

ECS DOTS: когда стандартный Unity сдаётся

Серия «ECS DOTS на практике», статья 1 из 4 — концептуальное введение

Контекст

Делал pet-проект — top-down survivors клон с целью **500+ врагов одновременно на экране** при таргете 60 FPS на мобильном устройстве. Стандартный MonoBehaviour-подход начал заваливаться уже на 80–100 врагах: GC-спайки, дёрганный фрейм-тайм, перегретое устройство через минуту игры.

Решением стал переход на **ECS DOTS** — официальный Data-Oriented стек Unity. Эта серия из 4 статей разбирает что я понял в процессе и как это применять в production.

В первой статье — **фундаментальное «почему»**. Без кода, без туториалов. Что делает ECS принципиально быстрее обычного Unity, и в каких ситуациях это того стоит.

Что такое ECS в трёх предложениях

- **Entity** — это просто целое число (ID). Никаких полей, никакого поведения.
- **Component** — структура с данными. Просто **struct**, без методов.
- **System** — функция, которая каждый кадр обрабатывает entities с заданным набором компонентов.

Всё. Никаких классов, наследования, виртуальных методов. Игра — это **трансформации над массивами данных**.

Если вы привыкли мыслить «Player наследуется от Character, который наследуется от Entity, и у каждого есть Update()» — придётся сломать привычку. В ECS такого нет.

Главный mental shift: данные отдельно от логики

Object-Oriented (MonoBehaviour) подход кладёт **данные и поведение в один класс**:

```
class Enemy : MonoBehaviour {
    public float health;      // данные
    public float speed;       // данные
    void Update() { ... }    // поведение
}
```

Каждый **Enemy** — managed-объект на куче. Данные одного зомби лежат рядом с его методами, vtable-указателем, ссылкой на Transform и десятком других служебных полей.

Data-Oriented подход разделяет:

```
// Где-то лежат все Health: [100][50][120][80][200] ...  
// Где-то лежат все MoveSpeed: [10][12][8][11][9] ...  
// Где-то лежит код, который обрабатывает обе колонки сразу
```

Это не косметическое изменение. Это **другая архитектура памяти**, которая позволяет процессору работать в режиме «массовой обработки» — за то же время сделать в 5–10 раз больше работы.

Дальше — почему именно так получается.

Memory Layout: AoS vs SoA

Это **ключ ко всему**. Понимание этой разницы — главное что нужно вынести из статьи.

Array of Structures (AoS) — то что делает MonoBehaviour

В памяти каждый объект **Enemy** лежит **целиком как блок**:

```
[Enemy 0: health, speed, transform, ai, audio, ... 200 байт]  
[Enemy 1: health, speed, transform, ai, audio, ... 200 байт]  
[Enemy 2: health, speed, transform, ai, audio, ... 200 байт]  
...
```

Когда система обработки урона хочет прочитать **только Health** у всех врагов, процессор тащит из памяти **200 байт на каждого**, чтобы выковырять 4 байта (**float**). Остальные 196 байт — мусор, который засоряет кэш.

Современный CPU читает память **cache line'ами по 64 байта**. На один cache line в AoS помещается **меньше одного целого объекта**. Чтобы пройти 500 врагов — 500+ cache misses подряд. Каждый cache miss = ~100 циклов простоя CPU.

Structure of Arrays (SoA) — то что делает ECS

В ECS компоненты одного типа лежат в памяти **поряд, отдельным массивом**:

```
Health:      [100][50][120][80][200][...500 значений...]  ← один непрерывный  
массив float  
MoveSpeed:   [10][12][8][11][9][...500 значений...]      ← другой массив  
LocalTransform: [...500 transform-ов подряд...]
```

Теперь когда система урона хочет прочитать **Health** всех врагов, она проходит **один плотный массив float'ов**. На 64-байтный cache line помещается **16 значений Health**. Один cache miss → 16 объектов обработано. CPU работает на полной скорости предсказания.

Результат: **на больших объёмах данных ECS в 5–20× быстрее** только за счёт этого. Никакой магии — просто памятью пользуются правильно.

💡 Если вы слышали словосочетание «Data-Oriented Design» — речь именно об этом. Эту идею сформулировал Mike Acton (insomniac Games) в знаменитом докладе на CppCon 2014. ECS DOTS — это Unity-имплементация этой идеи.

Cache Locality: что говорит производительность

Чтобы оценить масштаб, полезно держать в голове порядки величин (для современного x86/ARM):

Операция	Стоимость в циклах CPU
Регистр-регистр операция	1
L1 cache hit	~4
L2 cache hit	~12
L3 cache hit	~40
RAM (cache miss)	~100
Branch misprediction	~20
Virtual method call	~10–20
GC alloc + collect	сотни тысяч

MonoBehaviour-подход на 500 врагов даёт примерно: **500 cache miss × 100 циклов = 50 000 циклов** только на чтение, **до** какой-либо полезной работы. На 60 FPS бюджет одного кадра — ~16 миллисекунд = ~50 миллионов циклов. То есть **0.1% бюджета** уходит просто на промахи кэша.

С 5 системами, каждая обрабатывает данные → уже 0.5% впустую. Добавьте 2000 projectiles, 100 XP-гемов, particles... и кадр уже **состоит на 5–15% из cache miss'ов**.

ECS этот overhead убирает почти полностью.

Virtual Dispatch и GC — два других убийцы

Кроме памяти, MonoBehaviour несёт ещё две «налоговые» нагрузки:

Virtual dispatch на каждом Update()

`Update()` у `MonoBehaviour` — виртуальный метод. Unity проходит по всему списку активных скриптов и **через vtable вызывает** каждый Update. Каждый вызов — branch, который CPU может не предугадать. На тысячах объектов это десятки тысяч непредсказуемых переходов.

В ECS — **одна система = один проход по массиву данных**. Никаких vtable. Компилятор знает форму данных и может агрессивно оптимизировать.

GC pressure от managed-классов

Каждый `MonoBehaviour` — managed-объект. Создание, удаление, любая `new List<>()` — мусор для GC. **GC stop-the-world** длится **миллисекунды** и виден игроку как «фриз».

В ECS компоненты — **struct**-ы. Сидят в нативных непрерывных буферах под управлением **EntityManager**. **GC их не трогает в принципе**. Никаких аллокаций в горячем пути, никаких фризов.

Совокупно: убирая AoS + virtual + GC, ECS получает **5–20× ускорение** на массовых сценариях. Не на любых — на **массовых**. Об этом в следующем блоке.

SIMD и параллелизм — бонусом

Когда данные лежат плотно (SoA), процессор может применить **SIMD** — обработать 4 или 8 значений **за один такт**. Unity-стек включает в этот стек:

- **Job System** — раскидывает работу по worker-потокам, используя все ядра процессора параллельно
- **Burst Compiler** — AOT-компилирует C# job'ы в SIMD-нативный код (×3–10 поверх обычного **Mono.IL**)

Эти две темы — **статьи #3 и #4 серии**. В первой статье важно лишь зафиксировать: ECS архитектурно проектировался **под параллельную SIMD-обработку**. MonoBehaviour Update() — однопоточный, последовательный, не векторизуемый.

Когда брать ECS, а когда — нет

ECS — мощный, но **не бесплатный**. У него есть три налога:

1. **Кривая обучения**: ~2 недели полноценного onboarding'a для опытного Unity-разработчика
2. **Многословность**: больше boilerplate'a на простые задачи (Authoring + Baker + Component + System вместо одного MonoBehaviour)
3. **Совместимость**: Unity Physics → DOTS Physics, Animator → custom анимация, многие asset-store пакеты вообще не работают в ECS-мире

Решение «брать или нет» — это про **компромисс между этими налогами и выигрышем в производительности**.

Берите ECS	Не берите ECS
100+ однотипных объектов (враги, projectiles, particles)	UI и меню — оставляйте Mono + UI Toolkit
Жёсткие perf-таргеты: mobile, VR, Switch	Сложная логика на единственном объекте (GameManager)
RTS, survivors, factory games, swarm AI, симуляции	Прототип без perf-требований — overhead не окупится
В ТЗ слова «60 FPS на iPhone 8» / «1000 unit'ов одновременно»	Команда не готова инвестировать 2 недели в onboarding
Игра живёт долго и масштабируется (LiveOps, контент-патчи)	Тяжёлая зависимость от asset-store пакетов

Правило большого пальца: если можно показать «у нас будет 100+ X одновременно» — это серьёзный аргумент за ECS. Иначе сначала измеряйте профайлером, потом решайте.

Hybrid подход — реальность production

В моём pet-проекте структура такая:

- **90% кода — обычный MonoBehaviour:** UI, меню, метагейм (коллекция карт, BattlePass, фишинг)
- **10% — ECS DOTS:** только Survival-фича, где боль перформанса (массовые враги, projectiles, damage scan)

Между мирами — **бриджи**:

- Joystick (Mono, Canvas) пишет **MoveDirection** в ECS-singleton через статический класс
- HP-bar (Mono, World Space Canvas) читает **Health** и **MaxHealth** из ECS-entity игрока каждый кадр
- ECS damage-system эмитит **DamageEvent** entities, Mono-виджет в **LateUpdate** потребляет их и показывает floating «-5»

Это **самый частый production-сценарий**. Никто не переписывает весь проект на ECS — добавляют **точечно туда, где боль**. Моно-стек прекрасно справляется с UI и логикой меню, ECS закрывает массовку.

⚠ Что НЕ работает: попытка «уговорить» команду на полный переход с MonoBehaviour. ECS требует инвестиций, и без чёткого performance use-case'a инвестиция не окупится. Hybrid снимает риск.

Что дальше в серии

1. **ECS DOTS: когда стандартный Unity сдаётся** ← вы здесь
2. **ECS внутри: Components, Systems, Baker** — подробный разбор всех типов компонентов, паттернов Authoring → Baker, ISystem vs SystemBase, EntityCommandBuffer, типичные ошибки
3. **Jobs: параллелизм без race conditions** — IJob / IJobEntity, Schedule vs ScheduleParallel, ComponentLookup, race conditions и как их обходить
4. **Burst: магия нативного кода в C#** — что Burst компилирует, что ему нельзя, benchmark до/после на реальном проекте

Если делал хайлоад в Unity — поделись опытом в комментариях. Где ECS оправдан, а где это overkill?

References

- **Mike Acton — Data-Oriented Design and C++** (CppCon 2014). <https://www.youtube.com/watch?v=rX0ltVEVjHc> — фундамент всего что выше
 - **Unity DOCs — Entities package** <https://docs.unity3d.com/Packages/com.unity.entities@latest>
 - **Unity DOTS Sample** <https://github.com/Unity-Technologies/EntityComponentSystemSamples>
 - **Darkounity — What is Unity DOTS in 2026** <https://darkounity.com/blog/what-is-unity-dots-in-2026>
-

Описание картинки (для генерации)

Главная диаграмма для статьи — **AoS vs SoA**. Идея для генерации (Midjourney/ChatGPT image):

Two-column comparison diagram, programming/architecture style, clean technical illustration. Left column titled "MonoBehaviour (AoS)": vertical stack of identical boxes labeled "Enemy 0", "Enemy 1", "Enemy 2", each box contains coloured cells "health", "speed", "transform", "ai", "audio". Highlight one cell ("health") in each box with red outline — show that to read just "health" you must jump through full structs. Right column titled "ECS DOTS (SoA)": three separate long horizontal stripes labeled "Health array", "Speed array", "Transform array". Each stripe filled with same-coloured cells in a row. Highlight the entire "Health array" stripe with green outline — show contiguous memory access. At the bottom: small caption "Cache line: 64 bytes" with a horizontal bracket spanning ~16 cells on the SoA Health stripe and ~0.3 of one Enemy box on the AoS side. Style: flat design, two accent colors (red/green), technical but accessible, optimised for LinkedIn thumbnail viewing on mobile.

Альтернатива — взять готовую диаграмму с darkounity.com (с атрибуцией) или нарисовать в Figma / Excalidraw за 10 минут.